

RepositorioRemoto.Back

Práctica 6 · Servicio con almacenamiento local y remoto en .NET

Lucía Fuertes Cruz · Laura Santamaria Parra

Entorno Servidor · 2º DAW · Curso 2026-2027

API REST

EF Core

Caché

Docker

Tests

Gestión de usuarios con almacenamiento local y remoto

Un proyecto en .NET que conecta una API REST con una base de datos y una caché para gestionar usuarios de forma asíncrona.

Cada componente tiene una responsabilidad definida, lo que facilita ampliar la aplicación y comprobar su funcionamiento.

async

Result

simple · defendible · probado

Qué hemos construido

Un servicio de usuarios con tres piezas principales y una salida de exportación.

FLUJO PRINCIPAL



Lectura por ID: caché → BD → API.

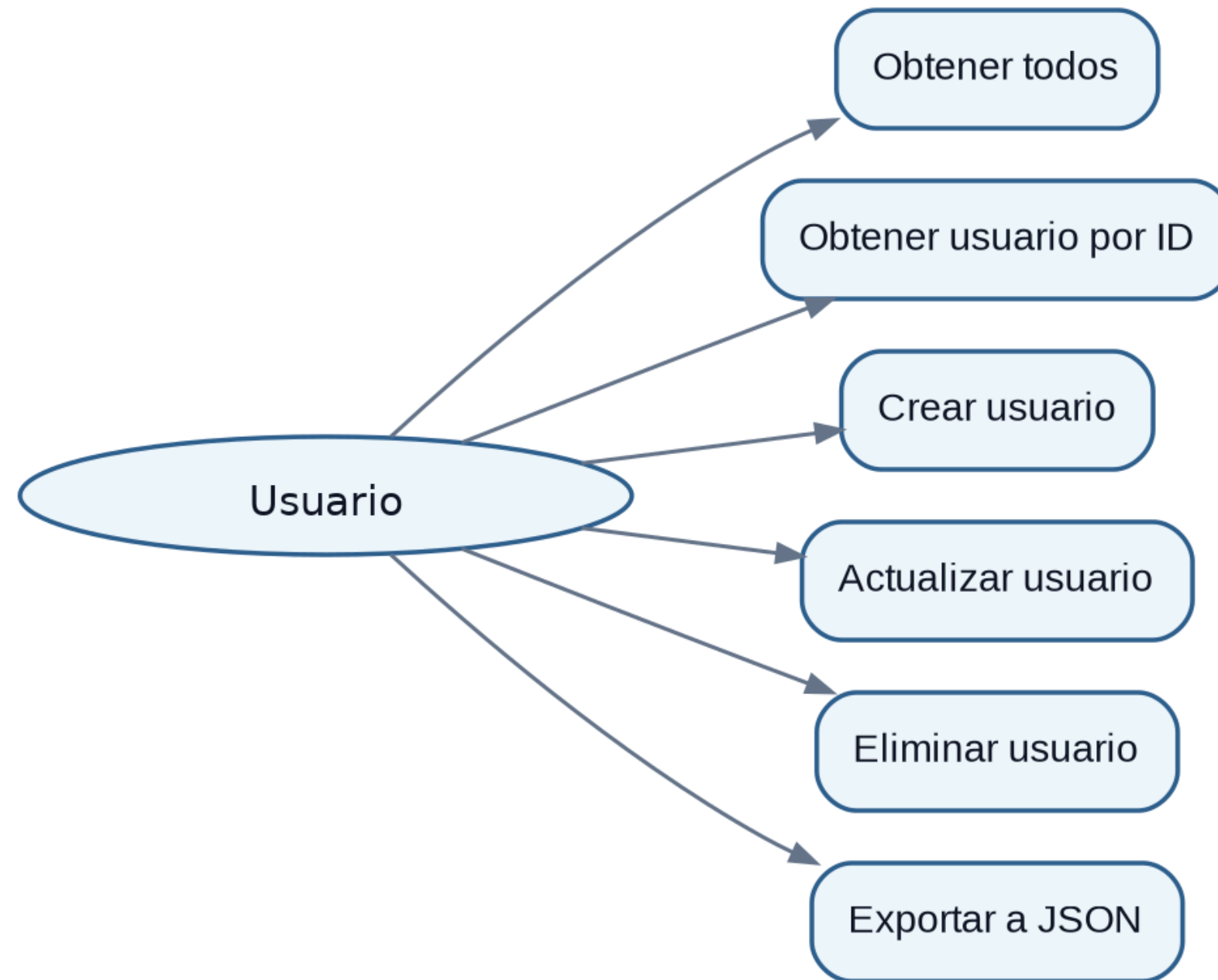
Escritura: API → BD → notificación.

GetAll no persiste automáticamente toda la lista al consultar API.

La exportación a JSON se genera bajo demanda.

Alcance funcional

Casos de uso principales centrados en el usuario.



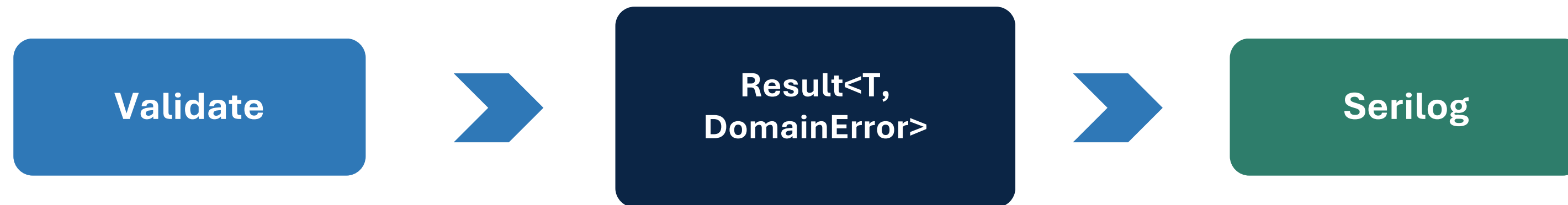
Las operaciones esenciales para gestionar usuarios

- Consultar todos los usuarios disponibles.
- Buscar un usuario por su identificador.
- Crear y actualizar usuarios con validación de datos.
- Eliminar usuarios y retirar su entrada de caché.
- Exportar los datos locales a un archivo JSON.

Usando `async` y `await` en cada caso correspondiente.

Validación, errores y exportación

El flujo evita usar excepciones como resultado normal de negocio.



RequestValidator comprueba campos obligatorios y regex.

UsersError, ServiceError, RepositoryError y StorageError separan fallos.

Serilog diferencia Debug, Information, Warning y Error.

UserStorage solo genera users.json bajo demanda; no es persistencia principal.

Inyección de dependencias con Scrutor

Una parte automática por interfaces marcadoras y otra manual para infraestructura.

AUTOMÁTICO

ITransientService
validadores

IScopedService
servicios y repos

ISingletonService
notificaciones/background

MANUAL

AddDatabase
SQLite/PostgreSQL

AddCache
Memory/Redis

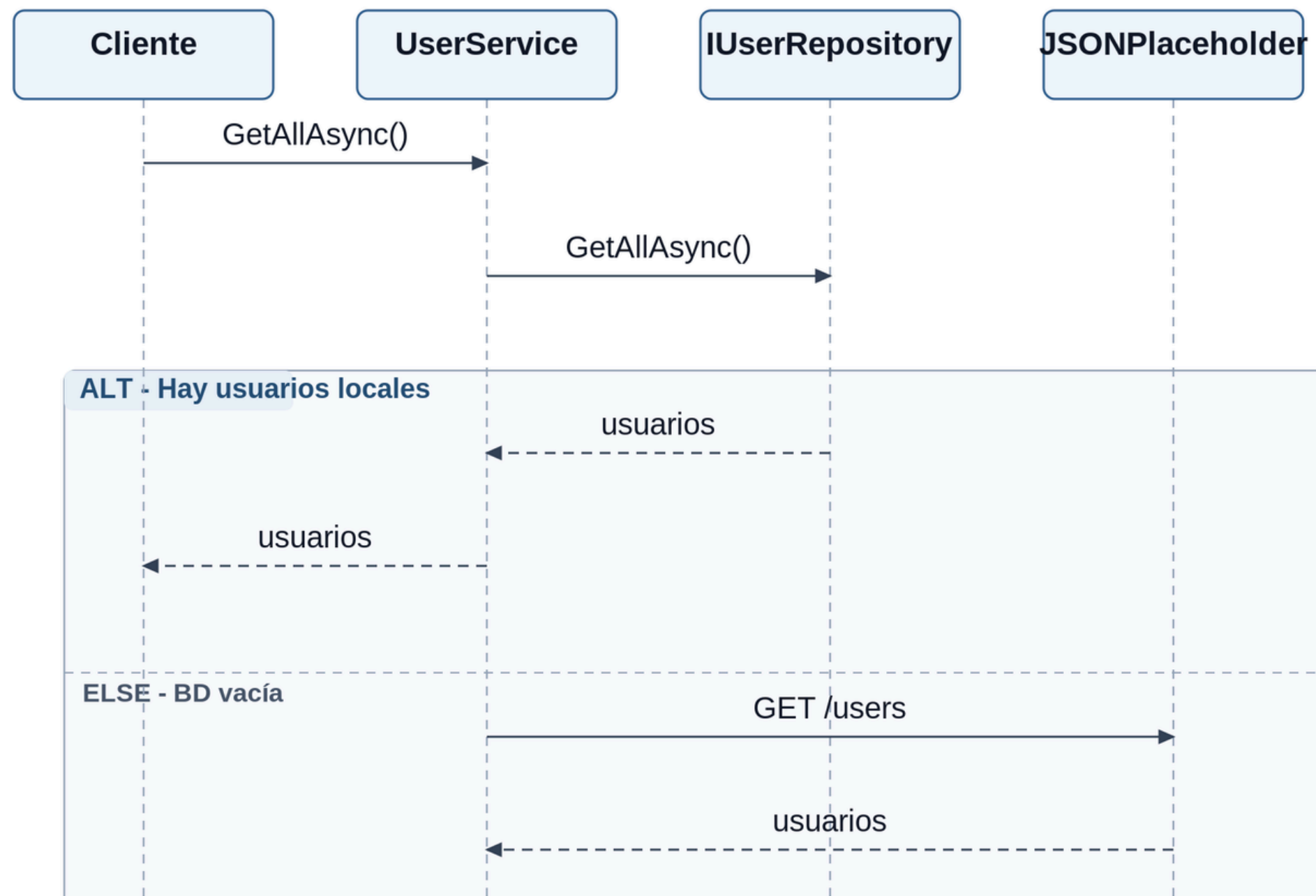
AddExternalApis
Refit

Resultado

Menos registro repetido.
Ciclos de vida visibles.
Infraestructura aislada.
Pruebas más fáciles con
mocks.

Lectura: GetAllAsync

La consulta general devuelve datos locales si existen; si la BD está vacía, consulta la API.

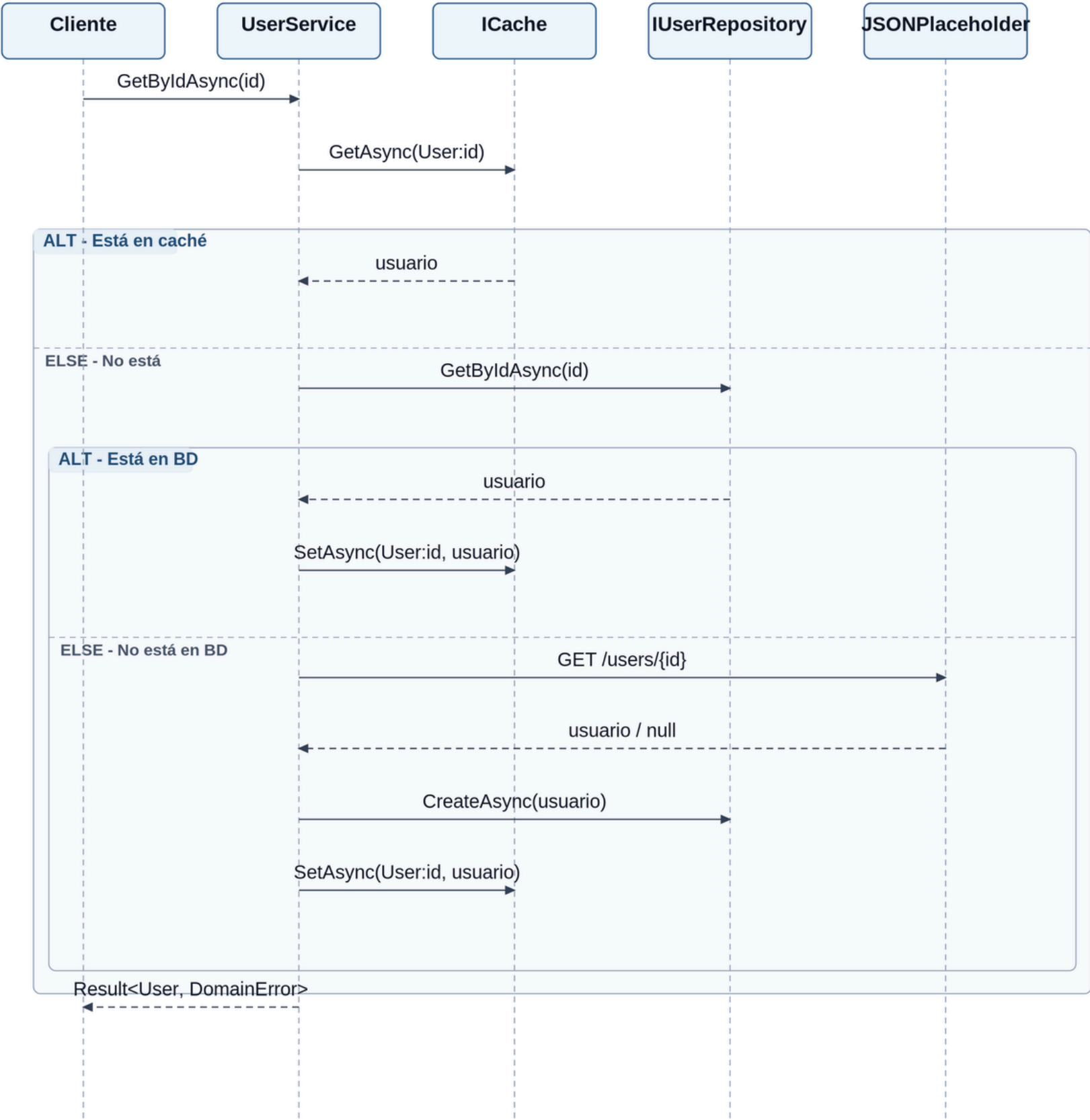


Lectura por ID

Aquí sí se aprovecha la caché para acelerar consultas repetidas.

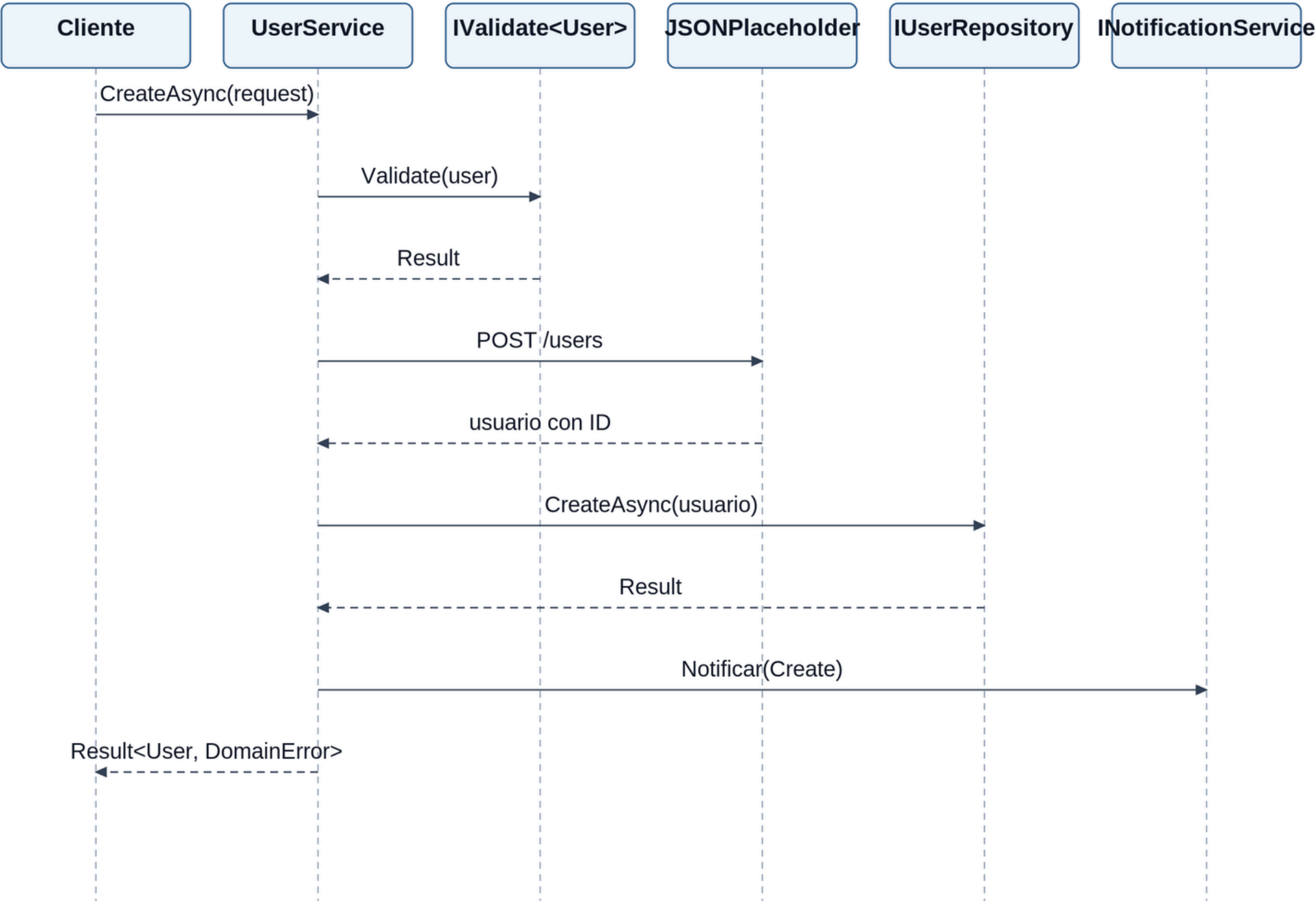
Orden de llamadas

- 1 · Caché
- 2 · Base de datos
- 3 · API REST
- Se cachea si viene de BD o API



Crear usuario

Validar → API → BD local → notificación.



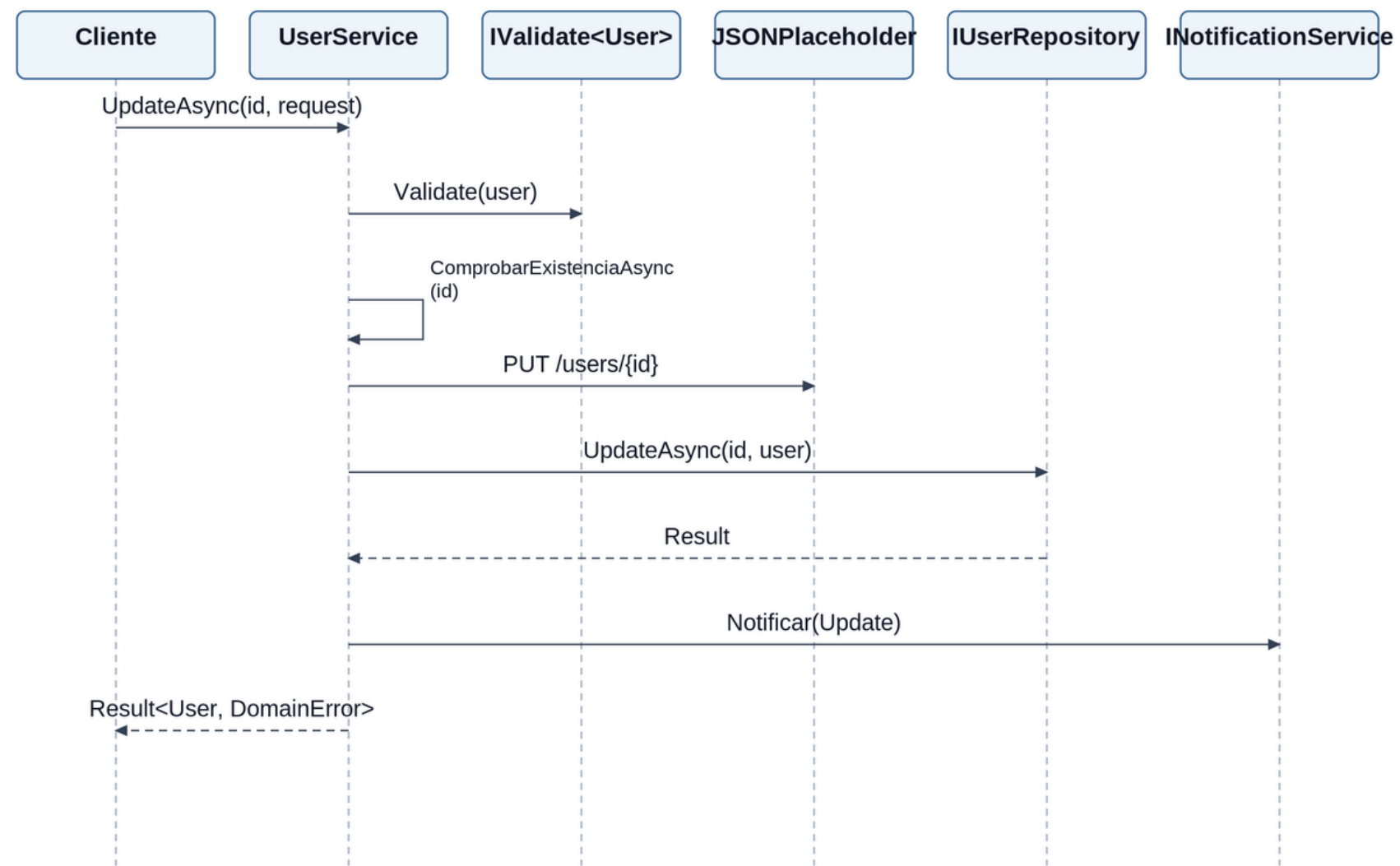
Decisión de diseño

Tanto en el caso de crear como el de actualizar se validan los datos y ya se guardan en la BBDD correspondiente.

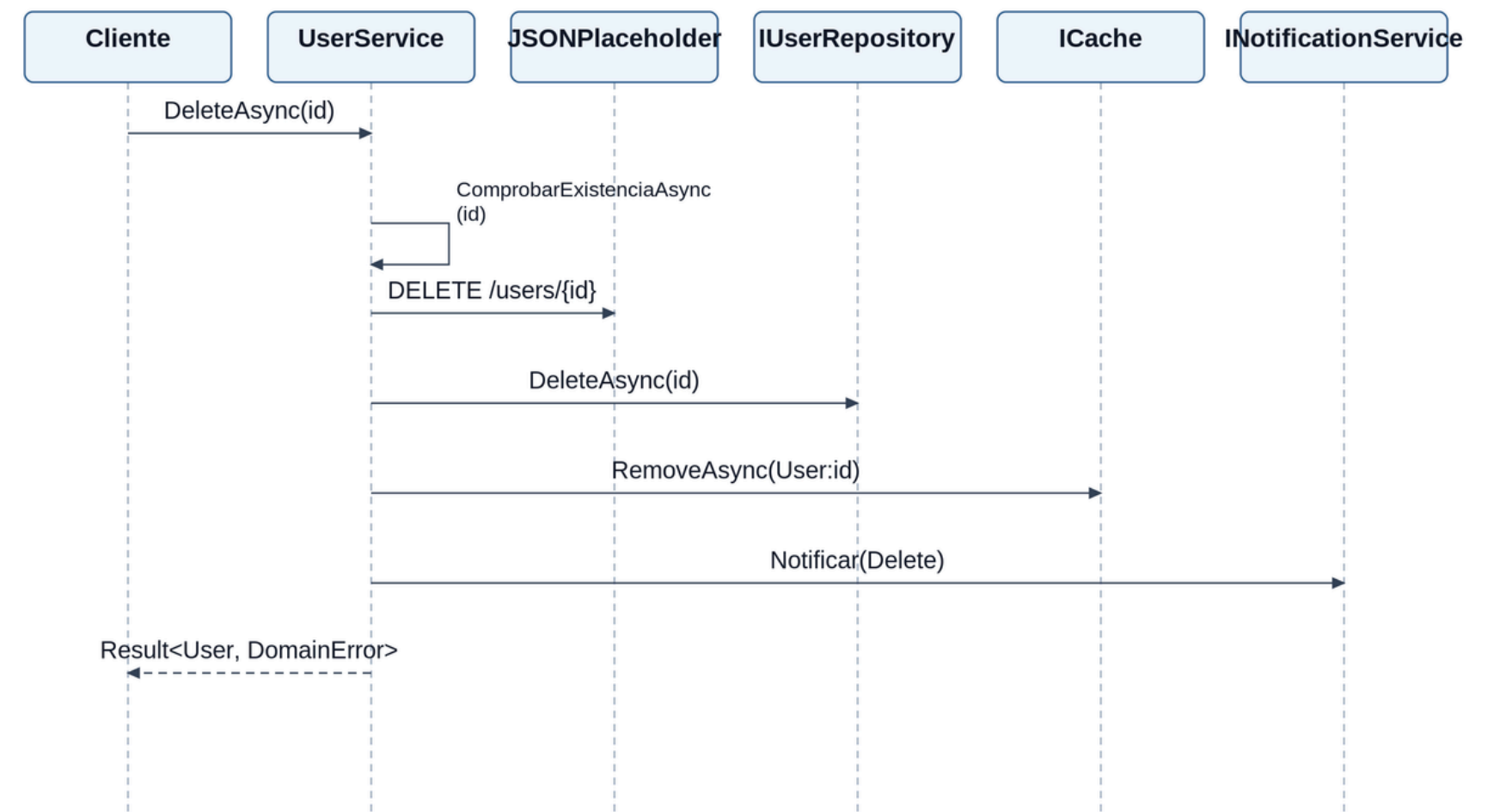
Al finalizar, se publica una notificación de creación. El usuario no se añade automáticamente a la caché, ya que todavía no se ha solicitado su consulta.

Actualizar y eliminar

Mismo enfoque: comprobar, operar y avisar.



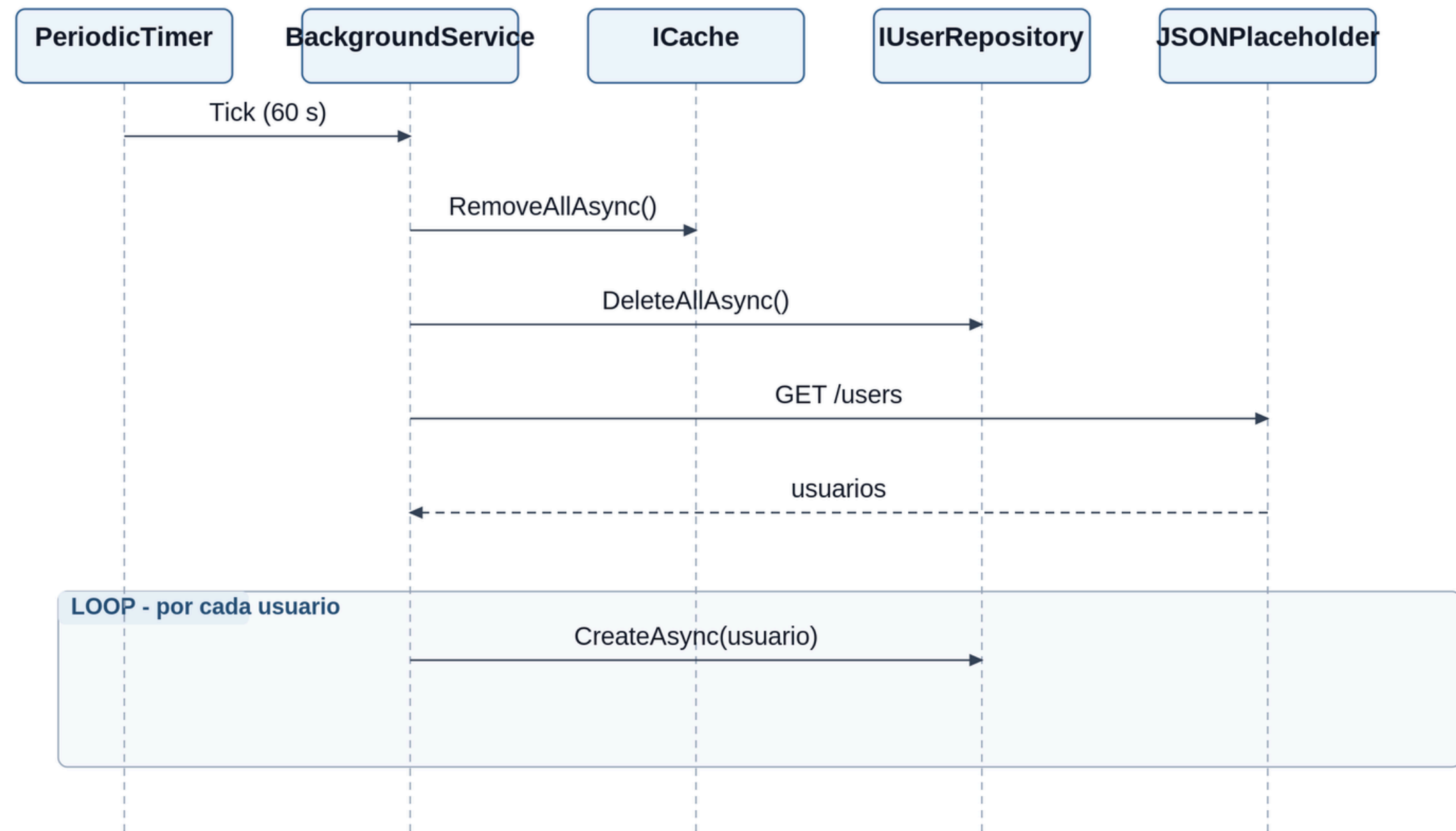
UpdateAsync: valida ID + datos + existencia



DeleteAsync: elimina también la clave de caché

Sincronización cada 60 segundos

BackgroundService limpia caché y recarga la copia local desde la API.



PeriodicTimer

RemoveAllAsync

DeleteAllAsync

GET /users

CreateAsync por usuario

Dos formas de ejecutar

La misma lógica cambia de infraestructura según el entorno.

Development

SQLite

MemoryCache

```
docker compose up -d --build
```

```
ENTRYPOINT ... Development
```

Production

PostgreSQL

Redis

```
docker compose --profile prod up -d --build
```

```
ENTRYPOINT ... Production
```

appsettings.development.json y appsettings.production.json cambian proveedores sin tocar UserService.

Pruebas y cobertura

Tests unitarios, integración con contenedores y revisión exacta de líneas.

NUnit

**Moq
+ Verify**

**Fluent
Assertions**

Testcontainers

**Report
Coverage**

Qué se comprueba

DTOs · modelos · mappers · DbContext · repositorios · caché Memory/Redis · BackgroundService · notificaciones · UserService · validadores · exportación JSON

Patrón AAA en los tests: Arrange → Act → Assert

Report Coverage se usa para saber qué líneas concretas quedan cubiertas.